

A modified ant colony system for solving the travelling salesman problem with time windows

Chi-Bin Cheng^{a,*}, Chun-Pin Mao^b

^a Department of Information Management, Tamkang University, 151 Ying-chuan Road, Tamsui, Taipei County, Taiwan

^b Cell Manufacturing Department, AU Optronics Corporation, 1 JhongKe Rd., Central Taiwan Science Park, Taichung, Taiwan

Received 11 September 2006; accepted 29 November 2006

Abstract

The travelling salesman problem with time windows (TSPTW) involves finding the minimum cost tour in which all cities are visited exactly once within their requested time windows. This problem has a number of important practical applications, including scheduling and routing. The problem is regarded as NP-complete, and hence traditional optimization algorithms are inefficient when applied to solve larger scale TSPTW problems. Consequently, the development of approximation algorithms has received considerable attention in recent years. Ant colony optimization (ACO), inspired by the foraging behaviour of real ants, is one of the most attractive approximation algorithms. Accordingly, this study develops a modified ant algorithm, named ACS-TSPTW, based on the ACO technique to solve the TSPTW. Two local heuristics are embedded in the ACS-TSPTW algorithm to manage the time-window constraints of the problem. The numerical results obtained for a series of benchmark problem instances confirm that the performance of the ACS-TSPTW is comparable to that of ACS-Time, an existing ACO scheme for solving the TSPTW problem.

© 2007 Elsevier Ltd. All rights reserved.

Keywords: Ant colony optimization (ACO); Travelling salesman problem with time windows (TSPTW); Meta-heuristic; Nature-inspired algorithm

1. Introduction

Ant colony optimization (ACO) is a meta-heuristic for solving hard combinatorial optimization problems. The first ACO algorithm, ant system (AS), was proposed by Colomni et al. [1] as a means of solving the travelling salesman problem (TSP). AS draws an analogy between the optimization process and the foraging behaviour of real ants. Based upon observations of real ant colonies, it was found that ants have the ability to find the shortest path between their nest and a food source. Furthermore, while walking back and forth between their nest and the food source, the ants deposit a chemical substance called *pheromone* along the paths they traverse. Different ants traversing the path at a later time sense the pheromone left by earlier ants and tend to follow the trail with a stronger pheromone concentration. Over a fixed time period, the shorter paths between the nest and the food source are likely to be travelled more often than the longer paths. Hence, the shorter paths accumulate a greater amount of pheromone, which, in turn, attracts more ants to the path, thereby further reinforcing the path.

* Corresponding author.

E-mail address: cbcheng@mail.tku.edu.tw (C.-B. Cheng).

Table 1
Applications of ant colony optimization (ACO)

Problem domain	Literature	Algorithm	year
Traveling salesman problem	Colormi et al. [1]	AS	1991
	Dorigo et al. [14]	AS	1996
	Gambardella and Dorigo [15]	Ant-Q	1995
	Dorigo and Gambardella [16]	ACS	1997
	Bullnheimer et al. [17]	ASrank	1999
	Stützle and Hoos [18]	MMAS	2000
Quadratic assignment problem	Gambardella et al. [3]	AS-QAP	1999
	Maniezzo [4]	ANTS-QAP	1999
	Stützle and Hoos [18]	MMAS-QAP	2000
	Talbi et al. [19]	Parallel Ant Colonies	2001
	Solimanpur et al. [20]	ACO	2004
Scheduling	Colormi et al. [9]	AS-JSP	1994
	Stützle [21]	AS-FSP	1998
	McMullen [10]	ACO	2001
	T'kindt et al. [11]	ACO	2002
	Gravel et al. [12]	ACO	2002
	Ying and Liao [13]	ACS	2004
	Shyu et al. [22]	ACO	2004
	Blum [23]	Beam-ACO	2005
Vehicle routing problem	Bullnheimer et al. [5]	AS-VRP	1999
	Gambardella et al. [24]	MACS-VRPTW	1999
	Bell and McMullen [25]	ACO	2004
Network routing	Schoonderwoerd et al. [7]	ABC	1996
	Di Caro and Dorigo [8]	AntNet	1998
Sequential ordering	Gambardella and Dorigo [26]	HAS-SOP	2000
Graph colouring	Costa and Hertz [6]	ANTCOL	1997
Constraint satisfaction	Solnon [27]	Ant-P-solver	2000
Classification	Shelokar et al. [28]	ACO classifier system	2004
Clustering	Shelokar et al. [29]	ACO	2004
	Kuo et al. [30]	Ant k -means	2005
	Yang and Kamel [31]	Multi-ant colonies	2006

AS solves the TSP using a principle similar to the foraging behaviour of a real ant [2]. Initially, agents (i.e. artificial ants) are placed at certain nodes in the network. Each agent then performs a series of random moves from one neighbouring node to another in accordance with a controlled transition probability or rule. In general, the transition probability along a particular arc depends on its degree of pheromone concentration and its length. After an agent has visited all of the nodes in the network, the quality of the tour is evaluated. The pheromone amounts on the paths traversed during the tour are then updated using predefined rules based on the quality of the tour. In solving the TSP using AS, the objective function of the optimization problem is defined in terms of the quality of the tour. When agents complete their tours, the tour with the best quality up to the current time point is memorized, and the agents repeat the forementioned procedures until stop criteria are satisfied.

The success of AS in solving the TSP has prompted its application to many other types of combinatorial optimization problem, including the quadratic assignment problem [3,4], vehicle routing [5], graph coloring [6], telecommunication network load balancing and distribution [7,8], and scheduling [9–13]. Table 1 presents a summary of previous applications of ACO to combinatorial optimization problems. Note that the intention of this table is only to provide an overall review of previous ACO applications in various problem domains; the contents are not intended to form an exhaustive list of such applications.

The structures of many combinatorial optimization problems comprise (or partially comprise) the TSP. However, some of these problems contain additional constraints, and hence they can not be solved directly using the original AS algorithm. Therefore, it is generally necessary to use additional local heuristics to guide the ant's search through the total set of feasible solutions [13]. Such local heuristics are problem-dependent [32] and are stored in the ant's

memory. For example, Bullnheimer et al. [5] extended AS to solve the vehicle routing problem (VRP) by incorporating standard VRP heuristics in the AS algorithm and using a tabu list to manage the tour length and capacity constraints.

The present study solves a particular variant of the TSP known as the travelling salesman problem with time windows (TSPTW) using a modified form of the ant colony system (ACS) proposed by Dorigo and Gambardella in [16]. The modified ACS is embedded with local heuristics designed to manage the time-window constraints of the problem. In the TSPTW, the salesman leaves the depot and must visit each customer node i during a particular time window $[a_i, b_i]$ before returning to the depot. Each arc (i, j) in the network has an associated travel time c_{ij} and each node i requires a particular service time, s_i . The objective of the TSPTW is to determine the minimum cost tour starting at the depot and passing through each customer node exactly once during their respective time windows. In establishing the optimum tour path, the salesman must arrive at node i no later than time b_i and can not start to serve the node until time a_i . However, waiting is permitted, i.e. the salesman can arrive at node i before time a_i . When applying ACS to solve the TSPTW, it is necessary to exclude those paths which violate the time-window constraints. In this study, this function is achieved via the introduction of two local heuristics.

2. Use of ACS in solving TSP

Given a graph $\mathcal{G} = (\mathcal{N}, \mathcal{A})$, where $\mathcal{N}$ is the set of nodes and $\mathcal{A}$ is the set of arcs, the objective of the TSP is to find the minimum length Hamiltonian circle on $\mathcal{G}$. ACS is an extension of the original AS algorithm and is designed to improve the efficiency of AS in solving the TSP.

2.1. Ant System (AS)

The AS algorithm defines the dynamics regarding the amount of pheromone deposited on each arc (i, j) , $\forall (i, j) \in \mathcal{A}$, as

$$\tau_{ij}(t) = (1 - \rho)\tau_{ij}(t - 1) + \Delta\tau_{ij}(t), \quad (1)$$

where $\tau_{ij}(t)$ is the amount of pheromone on arc (i, j) at time t , $0 < \rho < 1$ is the evaporation rate, and $\Delta\tau_{ij}(t)$ is the increase in the pheromone amount on arc (i, j) between time $(t - 1)$ and time t . Note that $\Delta\tau_{ij}(t)$ represents the accumulation of pheromone deposited by all of the ants which have travelled along the arc in this time period, i.e.

$$\Delta\tau_{ij}(t) = \sum_k \Delta\tau_{ijk}(t), \quad (2)$$

where $\Delta\tau_{ijk}(t)$ is the amount of pheromone laid by the k -th ant on arc (i, j) between time $(t - 1)$ and time t . Colomni et al. [1] proposed three different methods for determining the value of $\Delta\tau_{ijk}(t)$, namely ANT-density, ANT-quantity and ANT-cycle. Of these three methods, the authors reported that the ANT-cycle algorithm provided the best performance. According to this algorithm, the value of $\Delta\tau_{ijk}(t)$ after a complete tour is given by:

$$\Delta\tau_{ijk}(t) = \begin{cases} \frac{Q}{L_k}, & \text{if } (i, j) \in T_k, \\ 0, & \text{otherwise,} \end{cases} \quad (3)$$

where T_k denotes the tour completed by the k -th ant, L_k is the tour length and Q is a constant.

The transition probability governing the choice of the next node to visit, i.e. node j , by an ant at node i at time t is defined as:

$$p_{ij}(t) = \frac{[\tau_{ij}(t)][\eta_{ij}]^\alpha}{\sum_{z \in N_i} [\tau_{il}(t)][\eta_{iz}]^\alpha}, \quad j \in N_i, \quad (4)$$

where $N_i \subseteq \mathcal{N}$ is the set of all nodes in the neighborhood of node i which the ant has not yet visited; η_{ij} is a *visibility* term, defined as the inverse of the distance between node i and node j (i.e. $\eta_{ij} = 1/d_{ij}$), and α is a user-defined parameter which controls the relative importance of pheromone concentration versus visibility when evaluating the next node in the tour. The use of the visibility term enables the closer nodes to be chosen with a higher probability than more remote nodes. Note that this visibility factor is not observed in the foraging behaviour of real ants, but is introduced here as an artificial means of enhancing the efficiency of the optimization algorithm.

To ensure that an ant satisfies the Hamiltonian circle constraint when finding a tour, AS maintains a tabu list which records all of the nodes already visited by the ant during the current tour. Once the tour is completed, the tabu list is cleared and the ant is free to start a new tour with all of the nodes regarded as unvisited.

2.2. Ant colony system (ACS)

ACS differs from AS in three major ways [16]: (i) the state transition rule is modified to provide the ability to achieve a balance between exploring new arcs and exploiting accumulated knowledge about the problem, (ii) the global pheromone updating rule is applied only to arcs which belong to the best tour, and (iii) a local pheromone updating rule is applied whenever ants construct a tour.

2.2.1. State transition rule

The ACS state transition rule is composed of two separate strategies, namely exploitation and exploration. The exploitation strategy is a deterministic rule which always chooses the arc with the greatest product of pheromone intensity and visibility. Conversely, the exploration strategy is a stochastic rule with the form shown in Eq. (4). When determining which node to visit next, i.e. node j , the choice of strategy is made in accordance with the following prespecified random distribution

$$j = \begin{cases} \arg \max_{z \in N_i} \tau_{iz}(t) \cdot (\eta_{iz})^\alpha, & \text{if } q \leq q_0, \\ S, & \text{otherwise} \end{cases} \quad (5)$$

where q_0 ($0 \leq q_0 \leq 1$) is a user-defined parameter which specifies the distribution ratio of the two strategies, q is a randomly generated number uniformly distributed in the interval $[0, 1]$ and S is the transition probability rule defined in Eq. (4).

2.2.2. Global pheromone updating rule

In ACS, only those arcs which are contained in the shortest tour identified so far in the optimization process are assigned global pheromone. The purpose of this rule is to guide ants artificially toward the neighbourhood of the best tour found to date in the optimization process. The global pheromone updating procedure is performed once all of the ants have completed their tours in the current iteration. The amount of pheromone assigned to each arc in the shortest tour is given by:

$$\tau_{ij}(t) = (1 - \theta) \cdot \tau_{ij}(t - 1) + \theta \cdot \Delta\tau_{ij}(t), \quad (6)$$

where

$$\Delta\tau_{ij} = \begin{cases} (L_G)^{-1}, & \text{if } (i, j) \in T_G, \\ 0, & \text{otherwise} \end{cases}$$

$0 < \theta < 1$ is the global pheromone decay parameter, and L_G is the length of the globally best tour, T_G .

2.2.3. Local pheromone updating rule

After constructing a tour, the pheromone level is updated on each arc traversed by the ant during that tour in accordance with

$$\tau_{ij}(t) = (1 - \omega) \cdot \tau_{ij}(t - 1) + \omega \cdot \Delta\tau_{ij}(t), \quad (7)$$

where $0 < \omega < 1$ is the local pheromone decay parameter and the value of $\Delta\tau_{ij}(t)$ is either specified based on the results of Q -learning or is simply assigned the same value as the initial pheromone level. Note that the latter approach is adopted in this study.

3. Local heuristics and use of ACS in solving TSPTW

The schemes presented in the literature generally specify one of two different objectives when solving the TSPTW, namely to minimize the sum of the travel time on the path ignoring waiting time at the customer sites, or to minimize

the arrival time at the depot after completing the tour (i.e. the overall tour time) [33]. The present study adopts the latter objective since it is more realistic.

The time-window constraints imposed in the TSPTW problem can be expressed as

$$(t'_i + s_i + c_{ij})x_{ij} \leq b_j, \quad \forall (i, j), \quad (8)$$

where $x_{ij} \in \{0, 1\}$ is a decision variable with a value of 1 if arc (i, j) is travelled and 0 otherwise; $t'_i = \max\{a_i, t_i\}$, with t_i denoting the time the agent arrives at node i ; t'_i indicates the time point at which the agent can start to serve the node; and s_i is the service time at node i .

Solomon [34] pointed out that time-window constrained problems have both spatial and temporal aspects. To deal with the time-window constraints in the current TSPTW, two local heuristics are incorporated into the state transition rule used by the ants to choose their next move. In the original ACS algorithm, the visibility value, η , is introduced from the spatial aspect. However, in the modified ACS algorithm proposed in the current study, η is removed from the state transition rule because it is necessary to determine the total set of feasible solutions (i.e. paths which satisfy the time-window constraints) before evaluating the best tour. Thus, the temporal aspect is more emphasized in our approach. Nevertheless, the spatial aspect is still maintained through the pheromone updating rules, in which the updating procedure takes the tour length into account as well.

3.1. Local heuristics

In the algorithm developed in this study, two local heuristics are formulated to direct the ants towards appropriate nodes, i.e. nodes which satisfy two specific objectives, namely (i) to maximize the slack time in the traveled path such that the risk of violating the time-window constraints at later nodes in the trip is minimized and (ii) to minimize the waiting time in the event that waiting is unavoidable. The heuristics for these two objectives are denoted as g_{ij} and h_{ij} , respectively. The values of g_{ij} and h_{ij} are computed whenever an ant is required to choose its next destination node from the total set of unvisited nodes, i.e. $j \in N_i$.

The two heuristics are incorporated in the current algorithm by modifying the state transition rule in the original ACS algorithm given in Eq. (5) to the form:

$$j = \begin{cases} \arg \max_{z \in N_i} \tau_{iz}(t) \cdot (g_{iz})^\beta (h_{iz})^\gamma, & \text{if } q \leq q_0, \\ S, & \text{otherwise} \end{cases} \quad (9)$$

where β and γ are the relative importance parameters of g_{ij} and h_{ij} , and S is defined as:

$$P_{ij}(t) = \begin{cases} \frac{(\tau_{ij})(g_{ij})^\beta (h_{ij})^\gamma}{\sum_{z \in N_i} (\tau_{iz})(g_{iz})^\beta (h_{iz})^\gamma}, & j \in N_i \\ 0, & \text{otherwise.} \end{cases} \quad (10)$$

The heuristic g_{ij} is defined as follows. For an ant currently at node i , let t_j be the time at which it arrives at node j . The slack in the time window of node j is therefore given by $G_{ij} = b_j - t_j$. The rationale behind this heuristic is that the ant should visit those nodes whose arrival times are closer to their upper time-window constraints before those nodes with later upper time-window constraints in order to avoid the risk of lateness. In other words, the smaller the value of G_{ij} , the greater the value of g_{ij} . To enforce this principle, the following nonlinear relation based on a sigmoidal function is constructed between G_{ij} and g_{ij} :

$$g_{ij} = \begin{cases} \frac{1}{1 + \exp(\delta(G_{ij} - \mu))}, & \text{if } G_{ij} \geq 0 \\ 0, & \text{otherwise,} \end{cases} \quad (11)$$

where δ controls the slope of the function and μ is the inflection point, which is defined as the average of all $G_{ij} \geq 0$. The value of δ is assigned based on experimental results. The sigmoidal function in Eq. (11) is illustrated in Fig. 1. As shown, a node with greater G_{ij} is assigned a very low value of g_{ij} , and vice versa.

The heuristic h_{ij} is designed to reduce the amount of waiting time. For an ant at node i , if all of the potential arcs (i, j) , $j \in N_i$, will result in the ant waiting at node j , then the ant should choose the node with the least waiting time

when deciding which node to visit next. The waiting time at node j is given by $H_{ij} = a_j - t_j$. The relation between h_{ij} and H_{ij} is again defined using a sigmoidal function, i.e.

$$h_{ij} = \begin{cases} \frac{1}{1 + \exp(\lambda(H_{ij} - \nu))}, & \text{if } H_{ij} > 0 \\ 1, & \text{otherwise} \end{cases} \quad (12)$$

where λ controls the slope of the function and ν is the inflection point, which is defined as the average of all $H_{ij} > 0$. As shown, a node with a lower waiting time is assigned a higher value of h_{ij} , and is therefore more likely to be chosen as the next node in the path.

The local heuristics used in the current algorithm are similar to that proposed by Gambardella et al. [24] in their multiple ant colony system (MACS) designed to solve the vehicle routing problem with time windows (VRPTW). In their study, the authors divided the VRPTW into a set of TSPTW sub-problems, and used a module referred to as ACS-Time to solve each TSPTW. In the ACS-Time algorithm, the visibility term in the state transition rule is equivalent (approximately) to $(G_{ij} \cdot H_{ij})^{-1}$. However, in the current algorithm, the values of G_{ij} and H_{ij} are assigned different degrees of importance depending upon the particular problem under consideration. Furthermore, the respective degree of importance of each of these heuristics is further reinforced by their nonlinear treatment in Eqs. (11) and (12).

3.2. Use of ACS-TSPTW in solving TSPTW

The ACS algorithm formulated in this study, designated as ACS-TSPTW, is constructed by modifying the original ACS scheme proposed by Dorigo and Gambardella [16] via the addition of two local heuristics to the state transition rule. The basic processing steps in ACS-TSPTW are as follows:

- Step 0.** Initialization
 - Set $t = 0$
 - Set initial value $\tau_{ij}(t)$ for all $(i, j) \in \mathcal{A}$
 - Place m ants at the depot
 - Set $\Delta\tau_{ij}(t) = 0$ for all $(i, j) \in \mathcal{A}$
- Step 1.** Route Construction
 - For $i = 0$ to n do $\{n$ is the number of nodes; $i = 0$ denotes the depot $\}$
 - For every ant $k = 1$ to m do
 - If N_i not empty
 - Step 1.1** Local heuristics
 - Calculate g_{ij} and h_{ij} from Eqs. (11) and (12), respectively, $\forall j \in N_i$
 - If $g_{ij} = 0, \forall j \in N_i$, then give up the tour and go to next k $\{\text{no more feasible paths}\}$
 - Step 1.2** Move
 - Choose the node j to move to in accordance with the rules given in Eqs. (9) and (10)
 - Remove j from N_i
- Step 2.** Memorize the best tour found up to now, T^* , and its travel time, L^*
- Step 3.** Reset $N_i = \mathcal{N}$ for all ants
- Step 4.** Local Updating
 - For all $(i, j) \in \mathcal{A}$ do
 - Pheromone updating by Eq. (7)
- Step 5.** Global Updating
 - For all $(i, j) \in T^*$ do
 - Pheromone updating by Eq. (6)
- Step 6.** Stop Criterion
 - If the stop criterion is satisfied then stop, otherwise
 - Go to Step 1

In Step 0, the initial amount of pheromone on each arc is determined based on an approximate estimate of the tour length computed using the nearest neighbour heuristic proposed by Rosenkrantz et al. [35]. Dorigo and

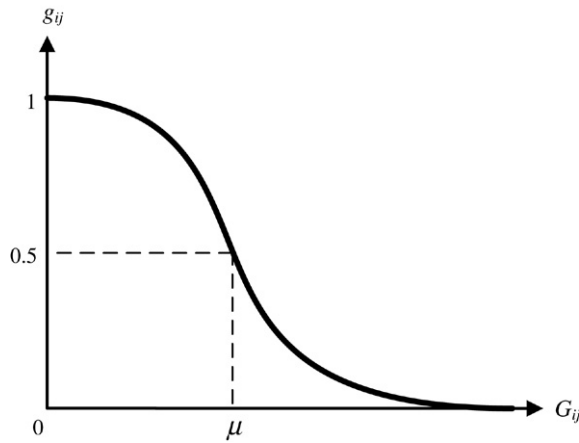


Fig. 1. Sigmoidal function.

Gambardella [16] suggested setting the initial amount of pheromone as $\tau_{ij}(0) = (n \cdot L_N)^{-1}$, where L_N is the length of the tour found using this nearest neighbor heuristic. In the global pheromone updating rule in Step 5, the term L_G in Eq. (6) is replaced with L^* , i.e. the travel time of the best tour so far since the objective of TSPTW is to minimize the total travelling time. The stop criterion in Step 6 is set as a maximal acceptable number of iteration to run the algorithm.

4. Numerical results

The performance of the proposed ACS-TSPTW algorithm was benchmarked using the thirty-one problem instances generated by Potvin and Bengio [36] from Solomon's RC2 VRPTW instances [34]. The nodes in the original problem instances were labelled with Cartesian coordinates and the distance between nodes can be obtained by computing their Euclidean distances based on these coordinates. However, to use these problem instances in the TSPTW setting, the distances between nodes are converted to travelling time between nodes.

Two groups of performance evaluation experiments were performed. In the first group, the performance of ACS-TSPTW was compared to that of ACS-Time [24]. The second group of experiments evaluated the quality of the ACS-TSPTW solutions by assessing their distance from the lower bounds of the corresponding problem instances. Since the optimum solution of many of the current problem instances is unknown, establishing the lower bounds of such problems provides a viable means of evaluating the quality of the ACS-TSPTW solutions.

4.1. Parameter settings of ACS-TSPTW

The parameter settings used in ACS-TSPTW have a direct effect on the algorithm's performance. These parameters include the number of ants (m), the distribution ratio parameter q_0 in the state transition rule, the pheromone decay parameters θ and ω , the relative importance parameters β and γ , and the slope parameters δ and λ .

Appropriate parameter values were determined by conducting an initial set of experiments using just one of the problem instances (rc206.3, comprising 25 nodes). In the experiments, the parameters were assigned different values and the problem was then solved repeatedly using the ACS-TSPTW algorithm with the value of one parameter being changed each time. The optimum combination of parameter values was specified as those values associated with the best experimental solution. The parameter values used in this initial set of experiments are shown in Table 2. The final parameter settings were determined to be $m = 3$, $q_0 = 0.99$, $\theta = \omega = 0.1$, $\beta = 0.5$, $\gamma = 3$ and $\delta = \lambda = 0.05$.

4.2. Performance comparison between ACS-TSPTW and ACS-Time

In comparing the performance of ACS-TSPTW with that of ACS-Time, each of the thirty-one problem instances in Potvin and Bengio [36] was solved five times using both algorithms. The two algorithms were run on a PC with

Table 2

Parameter settings used in parameter value determination experiments

Parameter	Values
m	3, 5, 10, 15
q_0	0.85, 0.9, 0.95, 0.99
θ	0.1, 0.3, 0.5, 0.9
ω	0.1, 0.3, 0.5, 0.9
β	0.5, 1, 3
γ	0.5, 1, 3
δ	0.01, 0.03, 0.05
λ	0.01, 0.03, 0.05

Table 3

Performance comparison

Problem instance	n	ACS-TSPTW			ACS-Time			RD	Lower bound
		Average	Best	CPU-time (s)	Average	Best	CPU-time (s)		
rc201.1	20	592.06	592.06	100.94	592.06	592.06	96.766	0.00	589.04
rc201.2	26	877.49	861.91	246.26	866.56	865.51	262.66	-0.42	839.62
rc201.3	32	867.61	853.71	464.3	854.11	854.09	466.13	-0.04	839.65
rc201.4	26	900.52	900.38	151.05	∞	∞	–	–	839.28
rc202.1	33	880.74	871.11	241.77	880.74	874.18	241.72	-0.35	839.62
rc202.2	14	338.52	338.52	46.765	382.47	338.52	47.172	0.00	338.52
rc202.3	29	892.18	847.31	190.24	∞	∞	–	–	839.28
rc202.4	28	∞	∞	–	874.55	856.37	170.75	–	839.65
rc203.1	19	673.07	663.66	78.828	600.66	572.63	80.438	15.90	431.30
rc203.2	33	926.75	897.88	255.77	911.34	902.35	278.96	-0.50	839.65
rc203.3	37	∞	∞	–	∞	∞	–	–	839.62
rc203.4	15	493.85	493.85	53.078	429.96	415.26	52.109	18.93	338.52
rc204.1	46	949.68	949.22	438.25	∞	∞	–	–	839.24
rc204.2	33	863.65	821.63	240.55	770.08	753.52	238.42	9.04	690.06
rc204.3	24	642.06	635.36	127.27	533.25	488.36	128.8	30.10	432.02
rc204.4	14	428.39	425.20	46.75	427.61	423.26	46.718	0.46	417.81
rc205.1	14	422.24	417.81	46.891	421.57	417.81	46.672	0.00	417.81
rc205.2	27	820.19	820.19	181.06	820.19	820.19	195.11	0.00	809.65
rc205.3	35	950.05	950.05	274.55	951.22	951.05	273.09	-0.11	899.24
rc205.4	28	870.43	850.99	186.563	849.32	838.75	180.18	1.46	834.62
rc206.1	4	117.85	117.85	13.265	117.85	117.85	13.406	0.00	96.01
rc206.2	37	914.99	909.30	306.13	906.98	905.47	304.27	0.42	719.62
rc206.3	25	650.59	650.59	140.72	650.59	650.59	140.51	0.00	622.08
rc206.4	38	943.31	943.31	320.19	∞	∞	–	–	719.65
rc207.1	34	860.98	851.06	258.44	889.33	875.21	258.28	-2.76	691.04
rc207.2	31	∞	∞	–	792.38	761.78	–	–	673.04
rc207.3	33	955.70	944.52	241.7	844.98	836.05	233.68	12.97	639.08
rc207.4	6	133.14	133.14	22.453	133.14	133.14	22.797	0.00	133.14
rc208.1	38	934.80	925.36	334.72	901.61	877.20	331.58	5.49	464.04
rc208.2	29	722.24	712.96	185.73	608.84	591.43	185.33	20.55	476.03
rc208.3	36	795.03	774.72	291.98	739.54	715.27	295.94	8.31	456.00

 n : Number of nodes in the problem; ∞ : Unable to find any feasible solution.

CPU of AMD Athlon 1.46 GHz. The best result (i.e. the shortest travel time of the tour) and the average result of the five replications are summarized in Table 3 for each instance.

The performance of our ACS_TSPTW algorithm was compared to the ACS_Time algorithm by using the following relative deviation (RD) index

$$RD = \frac{(ACS_TSPTW)^* - (ACS_Time)^*}{(ACS_Time)^*} \times 100\%,$$

where $(ACS_TSPTW)^*$ and $(ACS_Time)^*$ denote the best results obtained by ACS-TSPTW and ACS-Time, respectively. A RD less than 0 indicates that the ACS-TSPTW outperforms the ACS-Time. The computation time of the two algorithms are also reported in Table 3, where the columns of CPU-time record the computation time in second used by the two algorithms for each problem instance. It is seen that the computation time of the two algorithms is not significantly different.

As shown in Table 3, ACS-TSPTW fails to find a feasible solution for three problem instances, i.e. rc202.4, rc203.3 and rc207.2. However, TSP-Time fails to find feasible solutions for five instances, i.e. rc201.4, rc202.3, rc203.3, rc204.1 and rc206.4. From an inspection of the RD column, it is observed that ACS-TSPTW outperforms ACS-Time in six problem instances, i.e. rc201.2, rc201.3, rc202.1, rc203.2, rc205.3 and rc207.1, and achieves a comparable performance as ACS-Time in seven problem instances, i.e. rc201.1, rc202.2, rc205.1, rc205.2, rc206.1, rc206.3 and 207.4. Furthermore, ACS-TSPTW finds a feasible solution for four of the five problem instances for which ACS-Time fails. Overall, therefore, ACS-TSPTW outperforms ACS-Time in ten problems and achieves the same result in seven problems. In other words, the performance of ACS-TSPTW is better than or, at least as good as, that of ACS-Time in seventeen of the thirty-one problem instances.

4.3. Distance of ACS-TSPTW solutions from lower bounds of problem instances

The optimum solutions of many of the problem instances in Table 3 are unknown. Therefore, the accuracy of the ACS-TSPTW solutions can best be evaluated by considering their distance from the lower bounds of the corresponding problem instances.

The earliest time at which an agent can leave node i is $(a_i + s_i)$, where a_i is the lower limit of the time window (i.e. the earliest time at which the agent can start to serve the node) and s_i is the service time at the node. Ideally, due to the temporal nature of the problem, node j with $j = \arg \max_i \{a_i + s_i\}$ ought to be the final node, or at least to be among the final few nodes, to be visited in the optimum solution of the tour (if such a solution exists). In other words, the lower bound of the optimum solution is given by the sum of the value $(a_j + s_j)$ of such a node and the time spent traveling back to the depot from this node, i.e. (c_{j0}) . The lower bound computed using this approach for each problem instance is presented in the rightmost column of Table 3. For a given problem instance, any other feasible solution will yield a total travel time greater than or equal to $(a_j + s_j + c_{j0})$ because if j is not the final node, then, by definition, there must be at least one node after it, and hence the time spent travelling to these nodes and then back to the depot is clearly greater than c_{j0} . In general, it should be noted that the lower bound is not necessarily a feasible solution, but if it is feasible, then it must be the optimum solution.

Table 3 shows that for problem instances rc202.2, rc205.1 and rc207.4, the lower bounds are identical to the best solutions obtained by ACS-TSPTW and ACS-Time, respectively, which implies that the solutions obtained using these two algorithms are optimum solutions. Furthermore, for problem instances rc201.1, rc201.3, rc202.3 and rc205.2, the ACS-TSPTW solutions are close to the corresponding lower bounds. Therefore, there is a strong likelihood that these solutions are close to or even equal to the optimum solutions. The remainder of the problem instances are more complicated (i.e. they contain significant regions of overlapping time windows), and hence it is difficult to assess the quality of the ACS-TSPTW solutions via a comparison with the corresponding lower bounds.

5. Concluding remarks

This study has proposed an ant colony algorithm designated as ACS-TSPTW to solve the TSPTW. Two local heuristics have been formulated to manage the time-window constraints in the problem. The first heuristic is designed to maximize the accumulated slack time in the travelled path such that the risk of violating the time-window constraints at later nodes in the trip is minimized, while the second heuristic aims to reduce the waiting time at nodes in the event that waiting can not be avoided.

The performance of ACS-TSPTW has been compared to that of another ant colony algorithm, ACS-Time, in solving the thirty-one TSPTW problems presented by Potvin and Bengio [36]. The results have shown that ACS-Time fails to find feasible solutions for five of these instances. However, ACS-TSPTW finds feasible solutions for four of these five problems. Furthermore, ACS-TSPTW outperforms ACS-Time in six problems and obtains an equivalent

performance as ACS-Time in seven problems. Therefore, overall, the performance of ACS-TSPTW is better than or at least as good as that of ACS-Time in seventeen out of the thirty-one problem instances.

To enable a further assessment of the quality of the ACS-TSPTW solutions, the lower bounds of the thirty-one problem instances have been computed. The lower bound results have shown that ACS-TSPTW obtains the optimum solution in three of the problem instances. Furthermore, the ACS-TSPTW solutions of a further four of the problem instances are very close to the corresponding lower bounds, and hence have a high likelihood of being the optimum solutions.

References

- [1] A. Colomi, M. Dorigo, V. Maniezzo, Distributed optimization by ant colonies, in: *Proceedings of ECAL91 — European Conference on Artificial Life*, Paris, France, 1991, pp. 134–142.
- [2] W. Gutjahr, A graph-based ant system and its convergence, *Future Generation Computer Systems* 16 (2000) 873–888.
- [3] L.M. Gambardella, E. Taillard, M. Dorigo, Ant colonies for the quadratic assignment problem, *Journal of Operational Research Society* 50 (1999) 167–176.
- [4] V. Maniezzo, A. Colomi, The ant system applied to the quadratic assignment problem, *IEEE Transactions on Knowledge and Data Engineering* 11 (1999) 769–784.
- [5] B. Bullnheimer, R.F. Hartl, C. Strauss, An improved ant system algorithm for the vehicle routing problem, *Annals of Operations Research* 89 (1999) 319–334.
- [6] D. Costa, A. Hertz, Ants can colour graphs, *Journal of Operational Research Society* 48 (1997) 295–305.
- [7] R. Schoonderwoerd, O. Holland, J. Bruten, L. Rothkrantz, Ant-based load balancing in telecommunications networks, *Adaptive Behavior* (1997) 169–207.
- [8] G. Di Caro, M. Dorigo, AntNet: Distributed stigmergetic control for communications networks, *Journal of Artificial Intelligence Research* 9 (1998) 317–365.
- [9] A. Colomi, M. Dorigo, V. Maniezzo, M. Trubian, Ant system for job-shop scheduling, *Belgian Journal of Operations Research* 34 (1994) 39–53.
- [10] P.R. McMullen, An ant colony optimization approach to addressing a JIT sequencing problem with multiple objectives, *Artificial Intelligence in Engineering* 15 (2001) 309–317.
- [11] V. T'kindt, N. Monmarche, F. Tercinet, D. Laugt, An ant colony optimization algorithm to solve a 2-machine bicriteria flowshop scheduling problem, *European Journal of Operational Research* 142 (2002) 250–257.
- [12] M. Gravel, W.L. Price, C. Gagne, Scheduling continuous casting of aluminum using a multiple objective ant colony optimization metaheuristic, *European Journal of Operational Research* 143 (2002) 218–229.
- [13] K.-C. Ying, C.-J. Liao, An ant colony system for permutation flow-shop sequencing, *Computers and Operations Research* 31 (2004) 791–801.
- [14] M. Dorigo, V. Maniezzo, A. Colomi, The ant system: Optimization by a colony of cooperating agents, *IEEE Transactions on Systems, Man and Cybernetics* 26 (1996) 29–41.
- [15] L.M. Gambardella, M. Dorigo, Ant-Q: A reinforcement learning approach to the traveling salesman problem, in: *Proceedings of the 12th International Conference on Machine Learning, ML-95*, Tahoe City, CA, 9–12 July 1995, pp. 252–260.
- [16] M. Dorigo, L.M. Gambardella, Ant colony system: A cooperative learning approach to the traveling salesman problem, *IEEE Transactions on Evolutionary Computation* 1 (1) (1997) 53–66.
- [17] B. Bullnheimer, R.F. Hartl, C. Strauss, A new rank-based version of the ant system: A computational study, *Central European Journal for Operations Research and Economics* 7 (1) (1999) 25–38.
- [18] T. Stützle, H.H. Hoos, MAX-MIN ant system, *Future Generation Computer Systems* 16 (8) (2000) 889–914.
- [19] E.-G. Talbi, O. Roux, C. Fonlupt, D. Robillard, Parallel ant colonies for the quadratic assignment problem, *Future Generation Computer Systems* 17 (2001) 441–449.
- [20] M. Solimanpur, P. Vrat, R. Shankar, Ant colony optimization algorithm to the inter-cell layout problem in cellular manufacturing, *European Journal of Operational Research* 157 (2004) 592–606.
- [21] T. Stützle, An ant approach to the flow shop problem, in: *Proceedings of the 6th European Congress on Intelligent Techniques & Soft Computing, EUFIT'98*, Aachen, Germany, 7–10 September 1998, pp. 1560–1564.
- [22] S.J. Shyu, B.M.T. Lin, P.Y. Lin, Application of ant colony optimization for no-wait flowshop scheduling problem to minimize the total completion time, *Computers and Industrial Engineering* 47 (2004) 181–193.
- [23] C. Blum, Beam-ACO — Hybridizing ant colony optimization with beam search: An application to open shop scheduling, *Computers and Operations Research* 32 (6) (2005) 1565–1591.
- [24] L.M. Gambardella, É.D. Taillard, G. Agazzi, MACS-VRPTW: A multiple ant colony system for vehicle routing problems with time windows, in: D. Corne, M. Dorigo, F. Glover (Eds.), *New Ideas in Optimization*, McGraw Hill, London, UK, 1999, pp. 63–76.
- [25] J.E. Bell, P.R. McMullen, Ant colony optimization techniques for the vehicle routing problem, *Advanced Engineering Informatics* 18 (2004) 41–48.
- [26] L.M. Gambardella, M. Dorigo, Ant colony system hybridized with a new local search for the sequential ordering problem, *INFORMS Journal on Computing* 13 (3) (2000) 237–255.
- [27] C. Solnon, Solving permutation constraint satisfaction problems with artificial ants, in: *Proceedings of the 14th European Conference on Artificial Intelligence*, Berlin, Germany, 20–25 August 2000, pp. 118–122.

- [28] P.S. Shelokar, V.K. Jayaraman, B.D. Kulkarni, An ant colony classifier system: Application to some process engineering problems, *Computer and Chemical Engineering* 28 (2004) 1577–1584.
- [29] P.S. Shelokar, V.K. Jayaraman, B.D. Kulkarni, An ant colony approach for clustering, *Analytica Chimica ACTA* 509 (2004) 187–195.
- [30] R.J. Kuo, H.S. Wang, T.-L. Hu, S.H. Chou, Application of ant k -means on clustering analysis, *Computers and Mathematics with Applications* 50 (2005) 1709–1724.
- [31] Y. Yang, M.S. Kamel, An aggregated clustering approach using multi-ant colonies algorithms, *Pattern Recognition* 39 (2006) 1278–1289.
- [32] M. Dorigo, G. Di Caro, L.M. Gambardella, Ant algorithms for discrete optimization, *Artificial Life* 5 (1999) 137–172.
- [33] M. Gendreau, A. Hertz, G. Laporte, M. Stan, A generalized insertion heuristic for the traveling salesman problem with time windows, *Operations Research* 43 (3) (1998) 330–335.
- [34] M.M. Solomon, Algorithms for the vehicle routing and scheduling problems with time windows constraints, *Operations Research* 35 (2) (1987) 254–265.
- [35] D.J. Rosenkrantz, R.E. Stearns, P.M. Lewis, An analysis of several heuristics for the traveling salesman problem, *SIAM Journal on Computing* 6 (1977) 563–581.
- [36] J.-Y. Potvin, S. Bengio, The vehicle routing problem with time windows — Part II: Genetic search, *INFORMS Journal on Computing* 8 (1996) 165–172.